

Relatório 2 – Organização de Computadores I – INE5411

Alunas: Ana Luiza Sales Gobbi (24105453), Fernanda Bertotti Guedes (24100601) e Laísa Ágathe Bridi Dacroce (24100598)

Data: 27/04/25

1 INTRODUÇÃO

O presente relatório visa discorrer sobre os exercícios propostos no Laboratório 2 da disciplina de Organização de Computadores I. Os três exercícios propostos abordam a multiplicação de matrizes em Assembly. São apresentadas as soluções elaboradas, bem como as principais dificuldades encontradas.

2 EXERCÍCIOS PROPOSTOS

2.1 EXERCÍCIO 1

As matrizes A e B foram inicializadas na seção .data, cada uma como um espaço sequencial de memória contendo seus 9 elementos .word. Para armazenar a matriz resultante na memória de dados, foi criada uma label C .space, de maneira a alocar seu espaço, em bytes, no segmento de dados.

Ao realizar o cálculo do produto $A \cdot B^T$ à mão, foi possível perceber que não seria necessário obter a matriz B transposta via código, mas sim, alterar a lógica de multiplicação entre as matrizes A e B: cada linha de A seria multiplicada pelas três linhas (e não colunas) de B, para simular a multiplicação de A por B^T . Com essa alternativa, visamos obter mais agilidade na elaboração e execução do código, bem como poupar espaço de memória que seria alocado para a matriz transposta.

Com o objetivo de atestar o correto funcionamento do código, comparamos o resultado da matriz C presente no segmento de dados com a matriz obtida no cálculo à mão:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 0 & 0 & 1 \end{bmatrix} \quad B = \begin{bmatrix} 1 & -2 & 5 \\ 0 & 1 & -4 \\ 0 & 0 & 1 \end{bmatrix} \quad B^T = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 5 & -4 & 1 \end{bmatrix}$$
$$A \cdot B^T = \begin{bmatrix} 1 \cdot 1 + 2(-2) + 3(5) & 1(0) + 2(1) + 3(-4) & 1(0) + 2(0) + 3(1) \\ 0 \cdot 1 + 1(-2) + 4(5) & 0 \cdot 0 + 1(1) + 4(-4) & 0 \cdot 0 + 1 \cdot 0 + 4(1) \\ 0 \cdot 1 + 0(-2) + 1(5) & 0 \cdot 0 + 0(1) + 1(-4) & 0 \cdot 0 + 0 \cdot 0 + 1 \cdot 1 \end{bmatrix}$$
$$A \cdot B^T = \begin{bmatrix} 1 - 4 + 15 & 0 + 2 - 12 & 0 + 0 + 3 \\ 0 - 2 + 20 & 0 + 1 - 16 & 0 + 0 + 4 \\ 0 + 0 + 5 & 0 + 0 - 4 & 0 + 0 + 1 \end{bmatrix} = \begin{bmatrix} 12 & -10 & 3 \\ 18 & -15 & 4 \\ 5 & -4 & 1 \end{bmatrix}$$

Figura 1 – Cálculo manual de $A \cdot B^T$

Ao analisar a aba de Registradores do MARS, é possível perceber que os valores armazenados em \$s0, \$s1 e \$s2 correspondem, respectivamente, aos endereços-base das matrizes A, B e C, fato que pode ser confirmado, também, na coluna de endereços do Data Segment.

Registers		
Coproc 1		Coproc 0
Name	Number	Value
\$zero	0	0
\$at	1	0
\$v0	2	10
\$v1	3	0
\$a0	4	0
\$a1	5	0
\$a2	6	0
\$a3	7	0
\$t0	8	3
\$t1	9	3
\$t2	10	3
\$t3	11	1
\$t4	12	32
\$t5	13	268501096
\$t6	14	1
\$t7	15	1
\$s0	16	268500992
\$s1	17	268501028
\$s2	18	268501064
\$s3	19	3
\$s4	20	0
\$s5	21	0
\$s6	22	0
\$s7	23	0
\$t8	24	0
\$t9	25	0
\$k0	26	0
\$k1	27	0
\$gp	28	268468224
\$sp	29	2147479548
\$fp	30	0
\$ra	31	0
pc		4194472
hi		0
lo		6

Figura 2 – Registradores do exercício 1.

A imagem abaixo mostra os valores referentes à matriz A no segmento de dados (destacados em vermelho), os valores referentes à matriz B (destacados em azul) e, por fim, a matriz resultante C (em verde), com os mesmos valores obtidos no cálculo à mão.

Data Segment								
Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	1	2	3	0	1	4	0	0
268501024	1	1	-2	5	0	1	-4	0
268501056	0	1	12	-10	3	18	-15	4
268501088	5	-4	1	3	0	0	0	0
268501120	0	0	0	0	0	0	0	0
268501152	0	0	0	0	0	0	0	0
268501184	0	0	0	0	0	0	0	0
268501216	0	0	0	0	0	0	0	0
268501248	0	0	0	0	0	0	0	0
268501280	0	0	0	0	0	0	0	0
268501312	0	0	0	0	0	0	0	0
268501344	0	0	0	0	0	0	0	0
268501376	0	0	0	0	0	0	0	0
268501408	0	0	0	0	0	0	0	0
268501440	0	0	0	0	0	0	0	0

Figura 3 – Segmento de dados do exercício 1.

As principais dificuldades enfrentadas nesse exercício foram a compreensão do aninhamento de loops em Assembly, bem como o correto manejo dos índices i, j e k para percorrer as matrizes. A primeira, foi contornada com pesquisas na internet e consultas ao material disponibilizado pelo professor em slides, enquanto a segunda foi mitigada com a interpretação e escrita manual dos resultados a serem armazenados na matriz C.

2.2 EXERCÍCIO 2

Nesse exercício, a conversão dos valores da matriz resultante para a tabela ASCII se mostrou desafiadora. Porém, foi possível simplificar a lógica codificada, tendo em vista que

sabíamos o intervalo de valores que necessitavam de conversão, de modo que esta foi feita considerando apenas números de dois dígitos, positivos ou negativos.

Assim, foi necessário verificar se o número tinha um ou dois dígitos, por meio de uma instrução `blt`, para comparar com o valor 10. Então, para números com um dígito, bastaria somá-los ao valor 48, pois este indica o 0 na tabela ASCII. Para números com dois dígitos, uma divisão por 10 foi utilizada para separá-los em dois inteiros menores que 10 e converter cada um individualmente seguindo a mesma lógica de números com 1 dígito. Nessa porção do código, foram utilizados os conteúdos armazenados nos registradores `HI` e `LO` para tratar da conversão dos dígitos correspondentes ao resto e ao quociente da divisão, respectivamente.

No programa, o `buffer` - espaço reservado na memória para armazenar temporariamente os dados que seriam gravados no arquivo - foi usado para guardar os números da matriz `C` já convertidos para caracteres ASCII, junto com os espaços separando cada número. Conforme o programa calculava cada valor da matriz, ele ia preenchendo o `buffer`, aproveitando o loop já presente no código para calcular cada elemento de `C`. No final, todo o conteúdo do `buffer` foi escrito de uma vez no arquivo **“resultante.txt”**. Usar um `buffer` nos ajudou a organizar os dados e evitou a necessidade de fazer várias operações de escrita separadas, deixando o processo mais simples.

Após preencher todo o `buffer` com os caracteres correspondentes, foram utilizadas as `syscalls` específicas para abrir (`syscall 13`), escrever (`syscall 15`) e fechar (`syscall 16`) o arquivo, garantindo que o conteúdo fosse salvo corretamente no arquivo **“resultante.txt”**. Essas instruções para abertura/criação de arquivo, escrita em arquivo aberto e fechamento de arquivo foram baseadas no guia rápido do MARS disponibilizado pelo professor em slides.

Durante o desenvolvimento, uma dificuldade foi tentar usar diretamente os valores 32 (espaço) e 45 (sinal de menos) no código, sem precisar carregar da memória. No entanto, não foi possível fazer isso de forma simples como pensamos, porque o programa precisava usar registradores carregando esses caracteres como bytes prontos para salvar no `buffer`. Por isso, foi necessário declarar o espaço (“ ”) e o sinal de menos (“-”) na seção `.data` e carregar eles com `lb`, para depois armazená-los no `buffer`.

Após a execução do programa, o arquivo **“resultante.txt”** é gerado nos Downloads do computador, e, ao abri-lo, é possível encontrar a matriz `C` obtida com o produto entre `A` e `BT`. Conforme mostra a imagem abaixo, optou-se por imprimir `C` como uma única linha, para facilitar, tanto a implementação, quanto a comparação dos valores com a matriz resultante presente no segmento de dados.

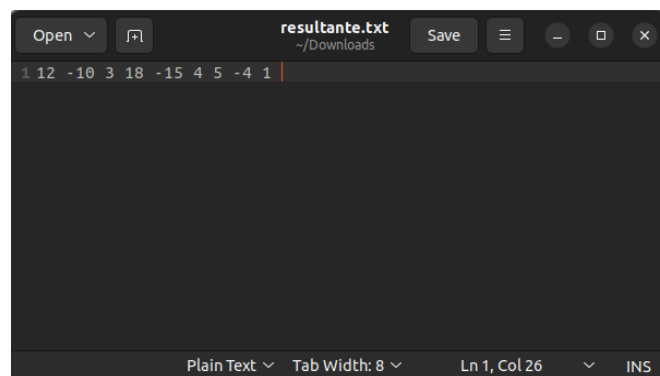


Figura 4 – Arquivo de texto com a matriz resultante `C`.

2.3 EXERCÍCIO 3

Em vez de carregar o endereço das matrizes A, B e C em registradores do tipo \$s, foram utilizados registradores do tipo \$a, para que estes valores fossem considerados argumentos disponíveis aos procedimentos criados. Adicionalmente, foi necessário reservar um espaço no segmento de dados para a matriz B transposta, com o mesmo tamanho reservado à matriz C: 36 bytes. Para acessar ambos os procedimentos na main, utilizamos a instrução jal (jump and link) a fim de desviar para a label requisitada e salvar o endereço de retorno no registrador \$ra.

O procedimento PROC_TRANS recebeu como argumentos os registradores \$a0 e \$a1, os quais continham os endereços de memória da matriz B e do espaço armazenado para receber a matriz B transposta (BT), respectivamente. Nesse sentido, optamos por percorrer os elementos da matriz B indexada pelos índices i e j, e armazenar cada valor resgatado na posição BT[j][i], preenchendo, então, o espaço BT reservado na seção .data do programa.

Já no procedimento PROC_MUL, aproveitamos o código feito no exercício 1 para realizar a multiplicação das matrizes A e BT. Como já calculamos a matriz B transposta, a operação de multiplicação entre A e BT ocorreu da forma usual, ou seja, linha x coluna, e por isso, tivemos que adaptar a lógica dos índices do exercício 1 para esse exercício.

Na figura 5 abaixo, é possível observar que o conteúdo dos registradores \$a0, \$a1, \$a2 e \$a3 corresponde aos endereços das matrizes A, B, BT e C no segmento de dados. A figura 6 mostra as matrizes BT (destacada em verde) e C (destacada em laranja) com os valores corretos da matriz B transposta e o resultado da operação $A \cdot B^T$, o que comprova o bom funcionamento do código.

Registers	Coproc 1	Coproc 0
Name	Number	Value
\$zero	0	0
\$at	1	0
\$v0	2	10
\$v1	3	0
\$a0	4	268501028
\$a1	5	268501064
\$a2	6	268500992
\$a3	7	268501100
\$t0	8	3
\$t1	9	3
\$t2	10	3
\$t3	11	1
\$t4	12	32
\$t5	13	268501132
\$t6	14	1
\$t7	15	1
\$s0	16	3
\$s1	17	0
\$s2	18	0
\$s3	19	0
\$s4	20	0
\$s5	21	0
\$s6	22	0
\$s7	23	0
\$t8	24	0
\$t9	25	0
\$k0	26	0
\$k1	27	0
\$gp	28	268468224
\$sp	29	2147479548
\$fp	30	0
\$ra	31	4194352
pc		4194360
hi		0
lo		6

Figura 5 – Registradores do exercício 3.

Data Segment								
Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	1	2	3	0	1	4	0	0
268501024	1	1	-2	5	0	1	-4	0
268501056	0	1	1	0	0	-2	1	0
268501088	5	-4	1	12	-10	3	18	-15
268501120	4	5	-4	1	3	0	0	0
268501152	0	0	0	0	0	0	0	0
268501184	0	0	0	0	0	0	0	0
268501216	0	0	0	0	0	0	0	0
268501248	0	0	0	0	0	0	0	0
268501280	0	0	0	0	0	0	0	0
268501312	0	0	0	0	0	0	0	0
268501344	0	0	0	0	0	0	0	0
268501376	0	0	0	0	0	0	0	0
268501408	0	0	0	0	0	0	0	0
268501440	0	0	0	0	0	0	0	0

Figura 6 – Segmento de dados do exercício 3.